

ProgenEDA Proteus Generator Architecture

Removal-Only Donor Mutation, Validators, Beautifiers, Terminals, and Wiring Rules

1. Corrected core idea

The Proteus generator should not try to render full circuits from scratch. That path is fragile.

The generator should also not rely on cloning components, because cloning, reference renaming, synthetic record creation, and full rendering are all proven failure-prone methods in the current Proteus path.

The correct production method is:

```
Use an already-working donor circuit.  
Remove what is not needed.  
Move and beautify what remains.  
Then add terminals or wires only after validation.
```

The generator should first check whether a close removal-only donor exists.

A removal-only donor means:

```
The donor already contains all required components.  
The donor contains enough copies of each required component.  
The generator can produce the requested circuit by deleting extra components,  
terminals, wires, labels, or unused sections.  
No component cloning is required.  
No new IC component synthesis is required.  
No risky reference regeneration is required.
```

If a closest removal-only donor exists, use it.

If no close removal-only donor exists, then use the mega donor circuit.

If the mega donor still does not contain the required components or enough copies of them, then the generator should stop with a clean donor-missing error instead of attempting unstable cloning.

The main principle is:

```
Production path = delete-only first.  
No cloning in the main path.
```

No component synthesis in the main path.
No full render from empty project in the main path.

2. Corrected donor selection logic

The donor selector should not use this risky logic:

If repeated copies are needed, clone a single/family donor.
If only one or two components are needed, use a single-component donor.
If a template is known, use that template regardless of deletion fit.

The corrected logic is:

1. Search exact removal-only donors.
2. Search closest removal-only donors.
3. Search known template donors only if they can be reduced by deletion.
4. Search family donors only if they already contain the required quantity.
5. Search mega donor if no closer donor exists.
6. If no donor contains the required components and quantities, fail cleanly.

2.1 Exact removal-only donor

Use this when:

$\text{donor components} - \text{removable extras} = \text{requested components}$

This is the best case.

Example:

User asks for:

2x 74HC192

2x 7447

2x 7SEG

A donor already contains:

2x 74HC192

2x 7447

2x 7SEG

extra switches

extra LEDs

extra labels

Use this donor and delete extras.

2.2 Closest removal-only donor

Use this when there is no exact donor but a donor contains all required parts and only needs deletion.

Selection score:

- contains all required component types
- contains enough quantity of each type
- requires minimum number of deletions
- has same circuit family as request
- has previously passed open/compile validation
- has known pin/coordinate data
- has low historical failure count
- requires no cloning
- requires no unsupported ref rewrite

Best donor is the donor with the lowest deletion risk.

2.3 Template donor

A template donor is allowed only if it is still removal-only.

Example allowed:

User asks for one 74HC192 counter feeding 7447 and 7SEG.
Template donor contains two such chains.
Delete one chain.
Keep one chain.

Example not allowed:

User asks for three 74HC192 counters.
Template donor contains one chain.
Do not clone the chain.
Fail or use mega donor if mega contains three.

2.4 Family donor

A family donor is allowed only if it already has enough parts.

Example allowed:

```
User asks for 3x 74HC74.  
Family donor contains 6x 74HC74.  
Delete 3.
```

Example not allowed:

```
User asks for 3x 74HC74.  
Family donor contains 1x 74HC74.  
Do not clone.
```

2.5 Mega donor

The mega donor is the fallback donor bank.

It should contain many supported components and multiple copies of common components.

The mega donor is used when:

```
no exact donor exists  
no close removal-only donor exists  
no template donor fits  
no family donor fits  
mega donor contains all required components and enough copies
```

The mega donor should still be used through deletion, not cloning.

2.6 Clean donor failure

If no donor can satisfy the request through deletion, return:

```
E_DONOR_MISSING_REMOVAL_ONLY
```

The error should report:

```
requested components  
required quantities  
closest donor found  
missing component types  
missing quantities  
new donor needed
```

Example:

Cannot generate this request using the removal-only Proteus path.
Requested: 4x 74HC192, 4x 7447, 4x 7SEG.
Closest donor contains: 2x 74HC192, 2x 7447, 2x 7SEG.
Missing: 2x 74HC192, 2x 7447, 2x 7SEG.
Required action: create donor with at least 4 counter/display chains.

3. Supported component universe

This architecture is not only for the ICs shown in the screenshot. It should apply to every supported component class.

Supported or target-supported components include:

3.1 Basic digital ICs

74HC00
74HC02
74HC04
74HC08
74HC32
74HC86
74HC266

3.2 Larger digital / sequential / display ICs

74HC151
74HC157
74HC85
74HC283
4511
7447
74HC192
74HC76
74HC174
4027
7490
74HC74
74HC160

3.3 Discrete semiconductors

NPN
PNP

LED
diode, if supported
zener, if supported later

3.4 Analog ICs / timing ICs

LM741
NE555

3.5 Passives

RESISTOR
CAPACITOR
ELECCAP
INDUCTOR

3.6 Sources and terminals

DC voltage source
DC current source
AC voltage source, if supported
power terminal
ground terminal
input terminal
output terminal
clock input
reset input

The same rule applies to all of them:

Prefer removal-only donors.
Avoid cloning.
Avoid full rendering.
Fail cleanly if no donor can satisfy the request.

4. Full corrected pipeline

User prompt
↓
Request parser
↓

```
Component and circuit requirement extractor
↓
Removal-only donor selector
↓
Donor selector validator
↓
Deletion plan generator
↓
Deletion plan validator
↓
Proteus deletion mutator
↓
Placement validator
↓
User-request semantic validator
↓
Beautifier
↓
Beautifier validator
↓
Post-beautifier placement validator
↓
Post-beautifier semantic validator
↓
Pin/model database validator
↓
Terminal or wiring decision
↓
Terminal generator OR wiring planner
↓
Terminal validator OR route validator
↓
Wire drawer, if physical wiring selected
↓
Wire draw validator
↓
Wire beautifier
↓
Wire beautifier validator
↓
Final validator
↓
Manifest + generated Proteus output
```

5. Request parser rules

The request parser must extract:

```
requested component types
requested quantities
requested values
requested circuit family
requested connection intent
terminal mode or wire mode
simulation requirement
display/output requirement
clock/reset requirement
power/ground requirement
```

It must normalize aliases.

Examples:

```
74HC00 = NAND IC
7400 = NAND IC, if compatible donor exists
74HC08 = AND IC
4511 = BCD to 7-segment decoder/driver
7447 = BCD to 7-segment decoder/driver
electrolytic capacitor = ELECCAP
op amp = LM741, if user explicitly asks LM741 or generic op-amp substitution
is allowed
timer = NE555, if user explicitly asks 555 timer or generic timer
substitution is allowed
```

It must not silently substitute parts.

If substitution is needed, it must be explicit in the report.

6. Donor selector rules

6.1 Donor selector input

```
requested component list
requested quantities
requested family
requested circuit type
required mode: place-only / terminal / wired
supported donor index
donor history score
donor failure history
```


6.2 Donor selector output

```
selected donor
why it was selected
components to keep
components to delete
unsupported requirements
risk score
fallback reason, if mega donor selected
```

6.3 Donor selector validator

The donor selector validator must check:

```
selected donor exists
selected donor opens in Proteus manually/previously
selected donor contains all required component types
selected donor contains enough required quantities
selected donor does not require cloning
selected donor does not require unsupported component creation
selected donor has known component inventory
selected donor has deletion history or is marked deletion-test-ready
selected donor has no unresolved previous fatal bugs
```

Fail if:

```
required part missing
required quantity missing
donor requires cloning
donor inventory unknown
donor has known fatal corruption issue
donor is not removal-only capable
```

Error categories:

```
E_DONOR_NOT_FOUND
E_DONOR_INVENTORY_UNKNOWN
E_DONOR_MISSING_COMPONENT
E_DONOR_MISSING_QUANTITY
E_DONOR_REQUIRES_CLONING
E_DONOR_CORRUPTION_HISTORY
E_DONOR_NOT_REMOVAL_ONLY
```

7. Deletion plan rules

The deletion plan decides what to remove from the donor.

It must include:

```
components to keep
components to delete
wires to delete
terminals to delete
labels to delete
junctions to delete
text/model fragments to delete
dangling metadata to delete
```

Deletion must be conservative.

Rules:

```
Never delete a component marked keep.
Never delete a wire connected only between kept components unless the next
stage intentionally rewires.
Never leave orphan labels from deleted components.
Never leave orphan model/name text from deleted components.
Never leave invisible metadata for deleted components.
Never leave duplicate references after deletion.
Never leave model fields pointing to deleted parts.
Never leave dangling wire stubs unless marked intentionally.
```

Deletion validator checks:

```
deletion list only contains donor-existing objects
keep list and delete list do not overlap
all deleted component-linked text/model/name fields are included
all deleted component-linked wires are included or intentionally preserved
no required component is scheduled for deletion
no required terminal is scheduled for deletion
```

Error categories:

```
E_DELETE_PLAN_KEEP_DELETE_OVERLAP
E_DELETE_PLAN_REQUIRED_COMPONENT
E_DELETE_PLAN_UNKNOWN_OBJECT
```

```
E_DELETE_PLAN_ORPHAN_TEXT_RISK
E_DELETE_PLAN_ORPHAN_WIRE_RISK
```

8. Placement validator rules

The placement validator checks the project after deletion/component placement.

It must check:

```
all requested components exist
all required quantities match
no extra unsupported donor components remain
no duplicate references exist
every component has a valid model
every component has a valid reference
every component has a visible name/reference
every component model/name/value field is still linked to the component
component model is not blank
component value is valid where required
component is inside sheet bounds
component is not overlapping another component
component has a known donor identity
component has known size/bounding box
component has known pin map, if pin-supported
```

8.1 Current known bug-derived placement rules

The validator must explicitly check for current known errors:

```
No model specified for <component>
Duplicate part reference at <reference>
Missing model field
Blank model field
Deleted component left model field behind
Deleted component left reference text behind
Component moved but reference text did not move
Component moved but model/name text did not move
Component exists visually but has no simulation model
Component has duplicate internal ID
Component has duplicate visible reference
Component has mismatched donor type/model
```

8.2 History-derived validator rule

Every fixed bug must become a permanent validator rule.

Process:

1. A new generation bug occurs.
2. The bug is assigned a failure category.
3. The root cause is found.
4. A minimal reproducer donor/request is saved.
5. A validator rule is added to detect the bug before final output.
6. A regression test is added.
7. Future generations must fail early if the same pattern appears.

The validator must have a history rules file:

```
validator_history_rules.json
```

Each rule should contain:

```
bug_id
first_seen_date
error_message
root_cause
validator_check
failure_category
regression_case
status
```

Example:

```
bug_id: PVAL_001
error_message: No model specified for U3
root_cause: donor deletion/move left component without valid model field
validator_check: every component must have non-empty model field matching
donor registry
failure_category: E_COMPONENT_MODEL_MISSING
```

Example:

```
bug_id: PVAL_002
error_message: Duplicate part reference at U1
root_cause: donor retained duplicate ref after deletion/renaming
validator_check: visible references and internal part refs must be unique
failure_category: E_COMPONENT_REF_DUPLICATE
```

Error categories:

```
E_COMPONENT_MISSING
E_COMPONENT_EXTRA
E_COMPONENT_MODEL_MISSING
E_COMPONENT_MODEL_MISMATCH
E_COMPONENT_REF_DUPLICATE
E_COMPONENT_REF_BLANK
E_COMPONENT_REF_RESERVED
E_COMPONENT_NAME_TEXT_ORPHANED
E_COMPONENT_MODEL_TEXT_ORPHANED
E_COMPONENT_OUT_OF_BOUNDS
E_COMPONENT_OVERLAP
E_COMPONENT_INTERNAL_ID_DUPLICATE
E_COMPONENT_DONOR_ID_UNKNOWN
```

9. User-request semantic validator rules

This validator checks whether the generated circuit satisfies the user request.

It checks meaning, not just file structure.

Rules:

```
If user asked for 74HC192, 74HC192 must exist.
If user asked for 7447, 7447 must exist unless explicit substitution is
allowed.
If user asked for 4511, 4511 must exist unless explicit substitution is
allowed.
If user asked for 4 ICs, exactly 4 intended ICs must remain.
If user asked for LED output, LED components must exist.
If user asked for LM741, LM741 must exist and not be replaced by generic op-
amp.
If user asked for NE555, NE555 must exist and not be replaced by generic
clock source.
If user asked for NPN/PNP, transistor type must match.
If user asked for electrolytic capacitor, ELECCAP must be used, not generic
capacitor, unless allowed.
If user asked for resistor/capacitor/inductor values, values must match
normalized form.
If user asked for sources, correct source type must exist.
```

Substitution rules:

```
No silent substitution.
Substitution requires explicit policy.
```

```
Substitution must be written in manifest.  
Substitution must be visible in final warnings.
```

Error categories:

```
E_REQUEST_COMPONENT_NOT_SATISFIED  
E_REQUEST_QUANTITY_MISMATCH  
E_REQUEST_WRONG_VARIANT  
E_REQUEST_UNAPPROVED_SUBSTITUTION  
E_REQUEST_VALUE_MISMATCH  
E_REQUEST_SOURCE_TYPE_MISMATCH
```

10. Beautifier rules

The beautifier moves components and visual items to make the schematic clean.

It may move:

```
components  
reference labels  
model/name text  
value labels  
terminals  
section labels
```

It must not:

```
delete components  
add new components  
clone components  
change component model  
change component ref  
break component-label linkage  
leave name/model/value at old coordinate  
move only the symbol but not its text  
move only text but not the symbol  
change logical connection plan
```

10.1 Beautifier placement rules

General layout:

inputs/switches/buttons	far left
sources	far left or top-left
clock/reset controls	left/top-left
passive chains	left-to-right
transistors/op-amps/timers	center
logic gates	left-to-right by signal flow
counters/flip-flops/registers	center-left
mux/comparator/adder ICs	center
decoder/display drivers	center-right
LEDs/displays/meters	far right
power	top
ground	bottom

10.2 Component-specific layout rules

Passives

Resistors in series should align horizontally.
 Parallel branches should align vertically.
 Capacitors to ground should drop downward.
 Inductors in signal path should align with series direction.
 Electrolytic capacitors should keep polarity visible.

Sources

Voltage/current sources should be left or top-left.
 Positive side should face the main signal path when possible.
 Negative side should face ground rail when possible.
 Source labels/values must remain beside the source.

Logic gates

Inputs on left.
 Outputs on right.
 Gates arranged by logic depth.
 NOT gates close to the signal they invert.
 AND/NAND terms grouped together.
 OR/NOR output-combining gates placed after term gates.
 XOR/XNOR gates placed where bit operations happen.

ICs

ICs should be aligned by function.
 Same-family ICs should align in a row or column.
 Counter chains should be ordered left-to-right or top-to-bottom.

Display drivers should be close to displays.
Muxes should be between input sources and destination.
Comparators/adders should have A/B inputs entering left and outputs exiting right.

Analog ICs

LM741 should have input side on left and output side on right.
Feedback components should stay near the op-amp.
NE555 should have timing RC components near its timing pins.
Transistors should be oriented consistently by role.
LED current-limiting resistors should stay near LEDs.

11. Beautifier validator rules

The beautifier validator is separate from the placement validator.

Its job is to detect visual and coordinate mistakes caused specifically by beautification.

It must check:

component symbol moved correctly
component reference text moved with symbol
component model/name text moved with symbol
component value text moved with symbol
label-to-component relative distance is still acceptable
model/name/value text is not left at old coordinate
component did not move outside sheet
component did not overlap another component after move
component did not overlap its own text
text label did not overlap another component
pin anchor positions updated after movement
bounding box updated after movement
routing space still exists after movement
functional groups remain aligned
bus groups remain alignable

Specific bug example:

Beautifier moved U3 from x=300 to x=600.
But U3 reference label or model text stayed at x=300.
This must fail.

Specific checks:


```
For every moved component:  
  old_symbol_position != new_symbol_position  
  old_ref_text_position should also change  
  old_model_text_position should also change  
  text must remain within allowed distance from component
```

Allowed distance rule:

```
Reference text must remain near component body.  
Model/name text must remain near component body or in accepted hidden/field  
location.  
Value text must remain near component body for passives and sources.
```

Error categories:

```
E_BEAUTIFIER_REF_TEXT_NOT_MOVED  
E_BEAUTIFIER_MODEL_TEXT_NOT_MOVED  
E_BEAUTIFIER_VALUE_TEXT_NOT_MOVED  
E_BEAUTIFIER_TEXT_ORPHANED  
E_BEAUTIFIER_LABEL_OVERLAP  
E_BEAUTIFIER_COMPONENT_OVERLAP  
E_BEAUTIFIER_OUT_OF_BOUNDS  
E_BEAUTIFIER_PIN_ANCHORS_STALE  
E_BEAUTIFIER_ROUTING_SPACE_DESTROYED  
E_BEAUTIFIER_GROUP_ALIGNMENT_BAD
```

12. Pin and model database rules

Every supported component must have a registry entry.

Fields:

```
component type  
Proteus model name  
donor model identifier  
reference prefix  
allowed reference format  
body size  
label offsets  
pin names  
pin numbers  
pin local coordinates  
pin side  
pin direction/type
```

- pin electrical class
- simulation model policy
- hidden power policy
- unused pin policy
- value requirements
- polarity requirements

12.1 Digital IC pin classes

- input
- output
- bidirectional
- tri-state
- clock
- reset
- clear
- preset
- enable
- load
- select
- data
- power
- ground
- no-connect

12.2 Passive/discrete pin classes

- passive
- anode
- cathode
- collector
- base
- emitter
- drain
- gate
- source
- positive
- negative
- non-inverting input
- inverting input
- output
- supply positive
- supply negative
- trigger
- threshold
- discharge

```
control
reset
```

12.3 Model validation rules

```
Every component must have a model if simulation requires it.
Passives must have valid values.
Electrolytic capacitor polarity must be known.
LED polarity must be known.
NPN/PNP pin order must be known.
LM741 pin order and power pins must be known.
NE555 pin functions must be known.
Sources must have type and value/waveform.
Digital IC hidden-power policy must be known.
Analog parts must not be treated as simple digital gates.
```

Error categories:

```
E_MODEL_MISSING
E_MODEL_MISMATCH
E_MODEL_UNSUPPORTED
E_MODEL_VALUE_MISSING
E_MODEL_VALUE_INVALID
E_PIN_MAP_MISSING
E_PIN_DIRECTION_MISSING
E_POLARITY_UNKNOWN
E_HIDDEN_POWER_POLICY_UNKNOWN
E_SIM_MODEL_REQUIRED
```

13. Terminal generator rules

Terminal generator is used when terminal mode is selected or wiring fallback is needed.

It must:

```
place terminals near pins
name terminals cleanly
use human-readable signal names
avoid machine-like names
respect terminal direction
avoid duplicate illegal terminal labels
allow intentional same-net labels
keep global nets consistent
```

Terminal names should be:

```
CLK
RST
CLR
LOAD
EN
A0
A1
A2
A3
B0
B1
B2
B3
Q0
Q1
Q2
Q3
D0
D1
D2
D3
SEG_A
SEG_B
CARRY
BORROW
V0
G0
```

Reserved-name rules:

```
V0 only power terminal.
G0 only ground terminal.
VCC only supply.
GND only ground.
+5V only supply.
0 only ground/reference if supported.
No internal signal may use these names.
No output pin may drive these names.
```

Terminal validator checks:

```
all terminals exist
names are valid
directions are valid
reserved names are not misused
terminals do not overlap
```

```
terminal labels moved with terminals if beautified  
pin-to-terminal association is recorded
```

Error categories:

```
E_TERMINAL_MISSING  
E_TERMINAL_DUPLICATE_ILLEGAL  
E_TERMINAL_DIRECTION_WRONG  
E_TERMINAL_RESERVED_NAME_MISUSE  
E_TERMINAL_OVERLAP  
E_TERMINAL_LABEL_ORPHANED
```

14. Wiring planner rules

Wiring planner calculates routes only.

It must not draw wires into Proteus directly.

It outputs:

```
net name  
source pin  
destination pins  
wire segments  
junctions  
bus groups  
labels  
fallback labels  
route risk score
```

14.1 Wire geometry rules

```
Manhattan routing only.  
No diagonal wires.  
All wire corners should be sharp 90-degree bends.  
All wire endpoints must land exactly on pin/terminal anchors.  
All wire points should be grid-aligned.  
No zero-length segments.  
No duplicate same-net segments.  
No wire should pass through a component body.  
No wire should pass through a label.  
No unrelated nets should overlap.  
No accidental junctions.
```

14.2 Multi-pin net rules

Use trunk-and-tap routing.
Avoid spaghetti point-to-point routing.
Add junction dots where taps meet trunk.
Use backbone for CLK/RST/EN/LOAD.
Use rails or terminal strategy for V0/G0.

14.3 Bus rules

Detect and group:

A0-A7
B0-B7
Q0-Q7
D0-D7
S0-S3
SEG_A-SEG_G

Route buses as:

parallel
same spacing
same direction
bit order preserved
no crossings within group

14.4 Router-to-beautifier feedback

If wiring is blocked, router returns:

failed net
failed bus group
component causing obstruction
required channel width
old component coordinate
new suggested component coordinate
reason

Beautifier adjusts and reruns validators.

Maximum loop:

```
3 attempts
then fallback to labels/terminals
```

Error categories:

```
E_ROUTE_NO_CLEAR_PATH
E_ROUTE_COMPONENT_COLLISION
E_ROUTE_WIRE_COLLISION
E_ROUTE_BUS_TOO_TIGHT
E_ROUTE_NEEDS_RELAYOUT
E_ROUTE_MAX_ATTEMPTS
E_ROUTE_LABEL_FALLBACK
```

15. Route validator rules

Route validator checks the planned route before drawing.

It must verify:

```
all required nets routed or intentionally labeled
route starts at valid pin/terminal
route ends at valid pin/terminal
segments are horizontal/vertical
points are grid-aligned
no component intersection
no unrelated net collision
junctions are correct
bus order is correct
global nets use global strategy
wire length is reasonable
wire count is reasonable
```

Error categories:

```
E_WIREPLAN_INVALID_PIN
E_WIREPLAN_DISCONNECTED_NET
E_WIREPLAN_DIAGONAL_SEGMENT
E_WIREPLAN_ZERO_LENGTH
E_WIREPLAN_COMPONENT_INTERSECTION
E_WIREPLAN_NET_COLLISION
E_WIREPLAN_MISSING_JUNCTION
E_WIREPLAN_BUS_ORDER_ERROR
```

16. Wire drawer rules

Wire drawer mutates the Proteus file.

It must:

```
draw only validated route plans
not calculate routes
not move components
not rename nets
not delete components
not add components
save before/after files
record every inserted wire segment
```

Wire-draw validator checks:

```
expected wire count inserted
wire coordinates match plan
file size change plausible
file still structurally valid
no duplicate records inserted
no partial route inserted silently
```

Error categories:

```
E_WIRE_DRAW_INSERT_FAIL
E_WIRE_DRAW_COUNT_MISMATCH
E_WIRE_DRAW_COORD_MISMATCH
E_WIRE_DRAW_PARTIAL_INSERT
E_WIRE_DRAW_FILE_CORRUPT
```

17. Wire beautifier rules

Wire beautifier improves readability only.

Allowed:

```
merge collinear same-net segments
remove duplicate same-net segments
add junction dots
add readable labels
apply deterministic colors if supported
```



```
slightly move labels away from wires
group bus labels
```

Not allowed:

```
change topology
change pin connections
delete required wires
add new logical nets
move components
rename component refs
randomly rename nets
```

Color policy must be deterministic:

power/V0/VCC	red
ground/G0/GND	black or dark green
clock	blue
reset/clear	purple
enable/load	orange
data bus	default or grouped
display segment	consistent segment group
normal signal	default

Wire beautifier validator checks:

```
topology unchanged
wire count unchanged unless duplicate merge is reported
merged segments belong to same net
labels do not overlap components
labels do not create conflicting net names
colors follow net-class rules
```

Error categories:

```
E_WIRE_BEAUTIFY_TOPOLOGY_CHANGED
E_WIRE_BEAUTIFY_BAD_MERGE
E_WIRE_BEAUTIFY_LABEL_COLLISION
E_WIRE_BEAUTIFY_NET_RENAME_CONFLICT
E_WIRE_BEAUTIFY_COLOR_RULE_FAIL
```

18. Final validator rules

Final validator checks everything.

It must verify:

```
all requested components exist
all requested quantities match
no unsupported extras remain
all refs unique
all models valid
all values valid
all component labels correctly positioned
all terminals valid
all required wires or labels exist
all required nets connected
no reserved net misuse
no output drives power/ground
no obvious supply contention
no floating required input unless intentionally exposed
clock/reset handled for sequential ICs
unused inputs tied or marked
analog parts have model/value/power policy
passive values are valid
polarity-sensitive parts are oriented correctly
manifest exists
stage reports exist
```

Sequential-specific checks:

```
clock pin has source or terminal
reset/clear/preset/load pins have defined state or terminal
counter enable pins are handled
flip-flop J/K/D pins are not floating unless intentional
startup/reset warning exists if needed
```

Analog-specific checks:

```
LM741 has power pins handled
LM741 inputs/outputs not shorted illegally
NE555 timing pins are connected according to selected mode
NE555 reset/control pins handled
transistor pinout known
LED has current-limiting path warning if no resistor
electrolytic capacitor polarity known
source polarity/value known
```

Error categories:

```
E_FINAL_COMPONENT_MISSING
E_FINAL_EXTRA_COMPONENT
E_FINAL_REF_DUPLICATE
E_FINAL_MODEL_MISSING
E_FINAL_VALUE_INVALID
E_FINAL_CONNECTION_MISSING
E_FINAL_RESERVED_NET_MISUSE
E_FINAL_FLOATING_INPUT
E_FINAL_POWER_CONTENTION_RISK
E_FINAL_CLOCK_MISSING
E_FINAL_RESET_MISSING
E_FINAL_POLARITY_UNKNOWN
E_FINAL_ANALOG_MODEL_MISSING
E_FINAL_LABEL_ORPHANED
E_FINAL_FILE_INVALID
```

19. History-to-validator engine

Every bug becomes a future validator rule.

Required flow:

```
Bug occurs.
Bug is logged.
Bug is assigned an error category.
Root cause is found.
Minimal reproducer is saved.
Validator check is added.
Regression case is added.
Generator cannot pass future final validation if same issue exists.
```

History file:

```
validator_history_rules.json
```

Entry format:

```
{
  "bug_id": "PVAL_002",
  "first_seen": "2026-06-15",
  "message": "Duplicate part reference at U1",
  "stage": "placement_validator",
```

```
"root_cause": "Donor deletion left duplicate reference record",  
"validator_rule": "All visible and internal component references must be  
unique",  
"error_category": "E_COMPONENT_REF_DUPLICATE",  
"regression_case": "duplicate_ref_after_deletion_001",  
"status": "active"  
}
```

Known current history rules:

```
No model specified for <component>  
Duplicate part reference at <reference>  
Component moved but model/name text stayed at original location  
Component moved but reference stayed at original location  
Reserved net used as internal signal  
Floating sequential input  
Output connected to power/ground  
Missing hidden power policy  
Wrong requested component substituted
```

20. Final rule

The generator should prefer a smaller, known-good output over a bigger risky output.

The correct priority is:

1. Valid Proteus file
2. Correct requested components
3. No model/reference errors
4. Clean layout
5. Correct terminals
6. Correct wires
7. Beautiful wires

Never sacrifice file validity for visual beauty.

Never use cloning in the production path until cloning has its own proven donor, validator, and regression suite.

Never allow a bug to be fixed manually without turning it into a validator rule.